

DOCUMENTO DE REFERENCIA

Roles en el Desarrollo de Software

Explicación detallada del ecosistema profesional, desde la idea hasta la operación y la mejora continua



Producto · Diseño · Desarrollo · Arquitectura · Calidad · Datos · DevOps · Seguridad · Soporte

Preparado como material explicativo — incluye mapa visual del ecosistema

Índice general

1. Introducción
2. Negocio y producto
3. Diseño y experiencia de usuario
4. Desarrollo
5. Arquitectura y liderazgo técnico
6. Calidad
7. Datos e inteligencia artificial
8. Infraestructura y operaciones
9. Seguridad
10. Soporte y postlanzamiento
11. Cómo se conectan los roles
12. Relación entre los distintos roles
13. Roles que pueden superponerse
14. Clasificación general de los roles
15. Conclusión

Nota: en una empresa pequeña una persona puede cumplir varios roles a la vez; en una empresa grande estos roles suelen estar más especializados y repartidos entre distintos equipos.

1. Introducción

El desarrollo de software es una actividad multidisciplinaria. La creación de una aplicación no depende únicamente de programadores: también intervienen profesionales de negocio, producto, diseño, calidad, datos, infraestructura, operaciones, seguridad, soporte y liderazgo.

Todos estos perfiles trabajan sobre un mismo objetivo: transformar una necesidad o una idea en un producto útil, seguro, confiable, mantenible y capaz de evolucionar. El software ocupa el centro del ecosistema porque cada área aporta una perspectiva diferente sobre el mismo producto.

Las relaciones entre las áreas son continuas. Producto define prioridades; diseño organiza la experiencia; desarrollo implementa la solución; calidad verifica su comportamiento; infraestructura permite ejecutarla; seguridad protege los activos; soporte acompaña a los usuarios; y los equipos de datos aportan información para mejorar las decisiones.

El mapa siguiente resume las nueve grandes áreas del ecosistema y cómo se organizan alrededor del producto de software:

ROLES EN EL DESARROLLO DE SOFTWARE

Mapa general del ecosistema: desde producto hasta operaciones



Nota: en una empresa pequeña una persona puede cumplir varios roles; en una empresa grande suelen estar más especializados.

2. Negocio y producto

Esta área determina qué producto debe construirse, para quién se construye y qué problema debe resolver. Su misión es alinear las necesidades de los usuarios con los objetivos de la organización y convertirlas en prioridades comprensibles para los equipos técnicos.

2.1 Product Manager

Define la estrategia general y la visión del producto. Busca comprender el mercado, las necesidades de los usuarios y el valor que la solución debe ofrecer.

RESPONSABILIDADES

- Definir visión y objetivos del producto
- Investigar mercado y usuarios
- Priorizar oportunidades y funcionalidades
- Coordinar negocio, diseño y tecnología
- Definir métricas de éxito

PREGUNTAS CLAVE

- ¿Qué problema se desea resolver?
- ¿Quién utilizará el producto?
- ¿Qué funcionalidades aportan mayor valor?
- ¿Cómo se medirá el resultado?

Su función principal no consiste en programar, sino en orientar al equipo para que construya el producto adecuado.

2.2 Product Owner

Representa las necesidades del producto dentro del equipo de desarrollo y mantiene organizado el backlog.

RESPONSABILIDADES

- Priorizar historias de usuario y tareas
- Definir criterios de aceptación
- Aclarar requisitos al equipo
- Revisar avances y aceptar funcionalidades
- Mantener actualizado el backlog

El Product Manager se enfoca en la visión y la estrategia; el Product Owner transforma esa visión en trabajo concreto para el equipo.

2.3 Business Analyst

Analiza procesos y necesidades de la organización y los convierte en requisitos claros para el equipo técnico.

RESPONSABILIDADES

- Entrevistar usuarios y responsables
- Documentar requisitos funcionales
- Definir reglas de negocio
- Modelar procesos y flujos
- Servir de enlace entre usuarios y desarrolladores

Ayuda a evitar que el equipo construya una solución técnicamente correcta pero incorrecta desde el punto de vista del negocio.

2.4 Scrum Master / Agile Coach

Facilita el trabajo del equipo mediante prácticas ágiles. No es necesariamente el jefe del equipo: su función es mejorar la forma en que las personas colaboran.

RESPONSABILIDADES

- Facilitar reuniones y ceremonias ágiles
- Eliminar obstáculos del equipo
- Promover la colaboración y mejora continua
- Favorecer la autonomía del equipo

CEREMONIAS TÍPICAS

- Planificación del sprint
- Reunión diaria (daily)
- Revisión del sprint
- Retrospectiva

3. Diseño y experiencia de usuario

Esta área define cómo las personas comprenderán y utilizarán el producto. Un sistema puede funcionar técnicamente y, aun así, fracasar si resulta confuso, difícil de usar o no responde a las necesidades reales del usuario.

3.1 UX Researcher

Investiga a los usuarios para conocer sus necesidades, comportamientos, expectativas y dificultades.

RESPONSABILIDADES

- Realizar entrevistas y encuestas
- Observar el uso de productos y procesos
- Organizar pruebas de usabilidad
- Elaborar perfiles y mapas de experiencia
- Proporcionar evidencia para las decisiones de diseño

3.2 UX Designer

Diseña la experiencia general de uso. Busca que el producto sea comprensible, eficiente, coherente, accesible y útil.

RESPONSABILIDADES

- Crear flujos de navegación
- Elaborar wireframes y prototipos
- Organizar la información y definir recorridos
- Detectar puntos de fricción

Ejemplo: en una app de compras, analiza cuántos pasos necesita alguien para encontrar un producto, agregarlo al carrito y pagar.

3.3 UI Designer

Diseña la apariencia visual de la interfaz.

RESPONSABILIDADES

- Elegir colores y tipografías
- Diseñar botones, formularios e iconos
- Definir espacios, tamaños y jerarquías visuales
- Mantener coherencia entre pantallas y sistemas de diseño

El UX Designer se concentra en la experiencia y el recorrido; el UI Designer, en la presentación visual. Ambos trabajan de forma coordinada.

3.4 UX Writer / Content Designer

Diseña los textos que aparecen dentro del producto.

RESPONSABILIDADES

- Redactar botones, mensajes e instrucciones
- Mejorar mensajes de error
- Definir textos de formularios y notificaciones
- Mantener un tono coherente y reducir ambigüedades

Un buen mensaje de error no solo indica que algo falló: explica qué ocurrió y qué puede hacer la persona para resolverlo.

4. Desarrollo

El área de desarrollo transforma los requisitos y diseños en software funcional. Sus integrantes escriben, prueban, modifican y mantienen el código del producto.

4.1 Front-end Developer

Desarrolla la parte de la aplicación con la que interactúa directamente el usuario.

- Implementa interfaces web y las adapta a distintas pantallas
- Integra la interfaz con APIs y aplica validaciones visuales
- Optimiza accesibilidad y rendimiento

Tecnologías: HTML, CSS, JavaScript, TypeScript, React, Angular, Vue.

4.2 Back-end Developer

Implementa la lógica interna del sistema y los servicios que sostienen la aplicación.

- Crea APIs e implementa reglas de negocio
- Gestiona autenticación, permisos y bases de datos
- Controla errores, seguridad e integra servicios externos

Tecnologías: Java, Python, C#, Node.js, Go, PHP, Ruby, Kotlin.

4.3 Full-stack Developer

Trabaja tanto en front-end como en back-end.

- Crea interfaces y servicios; conecta bases de datos
- Implementa autenticación e integra componentes
- Participa en despliegues básicos

Frecuente en empresas pequeñas y startups. Ser full-stack no implica dominar todo al mismo nivel, sino trabajar en varias capas.

4.4 Mobile Developer

Crea aplicaciones para dispositivos móviles.

- Desarrolla apps Android, iOS o multiplataforma
- Integra cámara, GPS, sensores y notificaciones
- Optimiza batería, recursos y conectividad intermitente

Tecnologías: Kotlin, Swift, Flutter, React Native, .NET MAUI.

4.5 Game Developer

Desarrolla videojuegos y sus sistemas internos.

- Implementa mecánicas de juego, movimiento, física e IA
- Integra gráficos y sonido; optimiza rendimiento
- Desarrolla funciones multijugador y herramientas internas

Tecnologías: Unity, Unreal Engine, Godot, C#, C++.

4.6 Embedded / IoT Developer

Crea software que funciona dentro de dispositivos físicos.

- Programa microcontroladores y sensores
- Gestiona memoria, consumo energético y comunicación entre dispositivos
- Trabaja con equipos industriales, médicos o domésticos en tiempo real

Tecnologías: C, C++, Rust y Python en determinados entornos. Requiere comprender también hardware y comunicaciones.

5. Arquitectura y liderazgo técnico

Esta área toma decisiones técnicas de alto nivel, organiza la evolución del sistema y guía a los equipos de ingeniería.

5.1 Software Architect

Define la arquitectura general del sistema.

- Elige patrones arquitectónicos y separa el sistema en componentes
- Define cómo se comunican los servicios y selecciona tecnologías
- Analiza escalabilidad, rendimiento y tolerancia a fallos

Ejemplos: arquitectura monolítica, microservicios, orientación a eventos, servicios en la nube.

5.2 Tech Lead

Referente técnico cercano al trabajo cotidiano del equipo.

- Orienta a los desarrolladores y revisa código
- Resuelve problemas complejos y define buenas prácticas
- Coordina con arquitectura y producto

Su liderazgo es principalmente técnico y no siempre incluye funciones administrativas.

5.3 Engineering Manager

Coordina personas, equipos y procesos de ingeniería.

- Organiza equipos y objetivos; realiza seguimiento del trabajo
- Acompaña el desarrollo profesional y participa en contrataciones
- Resuelve conflictos y gestiona recursos

El Tech Lead se concentra en la dirección técnica; el Engineering Manager, en personas y organización. A veces ambos roles recaen en una misma persona.

6. Calidad

El área de calidad comprueba que el software funcione como se espera y ayuda a prevenir errores antes de que afecten al usuario. Debe acompañar todo el ciclo de desarrollo.

6.1 QA Manual

Realiza pruebas funcionales directamente sobre el producto.

- Crea casos de prueba y verifica requisitos
- Reproduce y documenta errores; confirma correcciones
- Valida navegadores y dispositivos

Ejemplos de prueba: campos vacíos, datos inválidos, límites, caracteres especiales, fallos de conexión, envíos duplicados.

6.2 QA Automation Engineer

Crea pruebas automáticas para reducir la repetición manual y detectar fallas con mayor rapidez.

- Automatiza pruebas de interfaz y APIs
- Crea pruebas de regresión e integra tests en pipelines
- Mantiene suites de automatización y analiza resultados

Herramientas: Selenium, Cypress, Playwright, Appium, Postman, JUnit, Pytest.

6.3 Performance Tester

Analiza el rendimiento del sistema bajo diferentes condiciones.

- Mide tiempos de respuesta y simula usuarios simultáneos
- Analiza consumo de CPU y memoria
- Ejecuta pruebas de carga y estrés; detecta cuellos de botella

Su objetivo es comprobar cómo se comporta el sistema cuando aumenta la demanda o los recursos están bajo presión.

7. Datos e inteligencia artificial

Esta área recolecta, organiza, analiza y utiliza datos para generar conocimiento, apoyar decisiones o construir sistemas inteligentes.

7.1 Data Engineer

Construye la infraestructura y los procesos que permiten transportar y preparar datos.

- Crea pipelines e integra fuentes de datos
- Limpia, transforma y automatiza cargas
- Diseña almacenes de datos y garantiza su calidad

7.2 Data Analyst

Transforma datos en informes y conclusiones útiles para la organización.

- Consulta bases de datos y crea tableros e informes
- Visualiza tendencias y compara indicadores
- Responde preguntas de negocio y presenta hallazgos

Herramientas: SQL, Excel, Power BI, Tableau, Python, R.

7.3 Data Scientist

Utiliza estadística, programación y aprendizaje automático para descubrir patrones y realizar predicciones.

- Prepara datos para modelado y entrena/compara modelos
- Crea sistemas de recomendación y detecta fraude
- Realiza clasificación y predicción

7.4 Machine Learning Engineer

Convierte modelos de inteligencia artificial en servicios utilizables en producción.

- Despliega modelos y crea servicios de predicción
- Automatiza entrenamiento y versiona modelos
- Monitorea resultados e integra modelos con aplicaciones

El Data Scientist suele investigar y crear el modelo; el ML Engineer logra que opere de forma estable, escalable y mantenible.

8. Infraestructura y operaciones

Esta área permite que el software se ejecute fuera de la computadora del desarrollador. Se ocupa de servidores, nube, automatización, despliegues, bases de datos, disponibilidad y operación cotidiana.

8.1 DevOps Engineer

Integra desarrollo y operaciones mediante automatización, colaboración y mejora continua.

- Crea pipelines de integración y entrega continua
- Construye y administra contenedores; infraestructura como código
- Mejora monitoreo, observabilidad y reduce tareas manuales

Herramientas: Git, GitHub Actions, GitLab CI, Jenkins, Docker, Kubernetes, Terraform, Ansible, Prometheus, Grafana. DevOps es también una cultura de colaboración, no solo herramientas.

8.2 Cloud Engineer

Diseña y administra recursos en plataformas de nube.

- Crea servidores, redes y almacenamiento; configura identidades y permisos
- Administra bases de datos y balanceadores; diseña copias de seguridad
- Controla costos y automatiza recursos

Plataformas: Amazon Web Services, Microsoft Azure, Google Cloud Platform.

8.3 Platform Engineer

Construye plataformas internas que permiten a los desarrolladores trabajar de forma más rápida y autónoma.

- Crea servicios de autoservicio y plantillas/estándares
- Centraliza métricas y registros; ofrece entornos reproducibles

Usa prácticas de DevOps, pero se concentra en construir una plataforma interna reutilizable por varios equipos.

8.4 Site Reliability Engineer (SRE)

Se ocupa de la confiabilidad de los servicios y del equilibrio entre innovación y estabilidad.

- Define indicadores y objetivos de servicio; automatiza operaciones
- Gestiona alertas e incidentes; analiza fallas
- Diseña tolerancia a fallos y mejora capacidad/disponibilidad

Conceptos clave: SLI (indicador de nivel de servicio), SLO (objetivo de nivel de servicio), SLA (acuerdo de nivel de servicio) y presupuesto de error.

8.5 System Administrator

Administra servidores, sistemas operativos y servicios.

- Instala y actualiza sistemas; crea usuarios y permisos
- Configura servicios y administra redes
- Supervisa recursos y gestiona copias de seguridad

Muchas tareas manuales hoy se automatizan mediante scripts y herramientas de gestión de configuración.

8.6 DBA (Database Administrator)

Administra las bases de datos y protege su disponibilidad e integridad.

- Instala y configura motores; gestiona usuarios y permisos
- Crea copias de seguridad y restauraciones; optimiza consultas
- Configura replicación y alta disponibilidad

Motores: PostgreSQL, MySQL, Oracle, SQL Server, MongoDB, Redis.

9. Seguridad

El área de seguridad protege el software, los datos, la infraestructura y los usuarios. La seguridad debe incorporarse desde las primeras etapas del desarrollo.

9.1 Cybersecurity / Security Engineer

Protege sistemas, redes e infraestructura.

- Analiza riesgos y gestiona accesos
- Detecta amenazas e investiga incidentes
- Aplica políticas, supervisa eventos e implementa cifrado

9.2 AppSec Engineer

Se especializa en la seguridad de las aplicaciones.

- Revisa código y detecta vulnerabilidades
- Analiza dependencias y realiza pruebas de seguridad
- Define prácticas seguras y capacita desarrolladores

Referencia habitual: metodologías de OWASP.

9.3 DevSecOps Engineer

Integra controles de seguridad dentro de los procesos DevOps.

- Automatiza análisis de código y escanea dependencias/contenedores
- Detecta secretos expuestos y evalúa infraestructura como código
- Firma y verifica artefactos

La idea central: la seguridad forma parte del pipeline, no depende solo de una revisión manual al final.

10. Soporte y postlanzamiento

El trabajo continúa después de publicar el producto. Es necesario acompañar a los usuarios, resolver incidentes, coordinar nuevas versiones y recoger información para futuras mejoras.

10.1 Support Engineer

Brinda soporte técnico y actúa como vínculo entre los usuarios y los equipos internos.

- Recibe consultas y analiza/reproduce problemas
- Consulta registros y orienta a usuarios
- Escala incidentes y documenta soluciones

10.2 Customer Success / Implementation Specialist

Customer Success ayuda a que el cliente obtenga valor del producto; Implementation se concentra en la puesta en marcha.

- Configura el producto y capacita usuarios
- Acompaña la adopción e integra sistemas
- Detecta necesidades de mejora y reduce el abandono

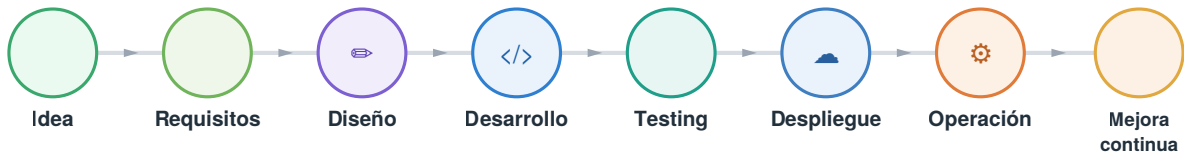
10.3 Release Manager

Coordina el lanzamiento de nuevas versiones.

- Organiza fechas y verifica dependencias
- Coordina equipos y valida aprobaciones
- Prepara notas de versión y gestiona planes de reversión

11. Cómo se conectan los roles

El ciclo de desarrollo puede representarse como una secuencia continua. Cada etapa involucra a diferentes especialistas, pero las decisiones se retroalimentan y el proceso vuelve a comenzar.



11.1 Idea

Comienza con una necesidad, oportunidad o problema. Participan Product Manager, responsables de negocio, usuarios y Business Analyst.

11.2 Requisitos

La idea se convierte en necesidades concretas y verificables. Participan Product Owner, Business Analyst, Product Manager, usuarios, arquitectos y equipo técnico.

11.3 Diseño

Se define cómo funcionará el producto y cómo será utilizado. Participan UX Researcher, UX Designer, UI Designer, UX Writer, Product Owner y desarrolladores.

11.4 Desarrollo

Los requisitos y diseños se convierten en código. Participan desarrolladores front-end, back-end, full-stack, mobile, data engineers, arquitectos y Tech Leads.

11.5 Testing

Se comprueba que el producto funcione y cumpla los requisitos. Participan QA Manual, QA Automation, Performance Tester, desarrolladores y AppSec.

11.6 Despliegue

La versión se publica en un entorno. Participan DevOps, Cloud Engineer, Platform Engineer, Release Manager, DBA y seguridad.

11.7 Operación

El sistema se mantiene disponible. Participan SRE, DevOps, System Administrator, DBA, Support Engineer y Security Engineer.

11.8 Mejora continua

Se analizan resultados, errores, opiniones y métricas. Todas las áreas aportan información para iniciar un nuevo ciclo.

12. Relación entre los distintos roles

Ninguna área trabaja de forma aislada. La calidad del producto final depende de la comunicación fluida entre estas parejas de disciplinas:

Producto y desarrollo — Producto define qué debe construirse y desarrollo determina cómo implementarlo. La comunicación evita requisitos ambiguos o inviables.

Diseño y desarrollo — Diseño define experiencia e interfaz; desarrollo las transforma en una aplicación funcional. Deben coordinar comportamiento, accesibilidad y restricciones técnicas.

Desarrollo y calidad — Desarrollo crea funcionalidades y calidad verifica su comportamiento. La calidad es una responsabilidad compartida.

Desarrollo y DevOps — Desarrollo produce código y DevOps automatiza su construcción, prueba y despliegue.

DevOps y seguridad — DevOps automatiza el flujo de entrega y seguridad incorpora controles. De esta relación surge DevSecOps.

Operaciones y soporte — Operaciones mantiene el sistema funcionando y soporte recoge problemas de los usuarios.

Datos y producto — Datos analiza el comportamiento del sistema y producto utiliza esa información para decidir mejoras.

13. Roles que pueden superponerse

La distribución de funciones cambia según el tamaño y la madurez de la organización.

13.1 Empresa pequeña

- Un Full-stack Developer trabaja en front-end y back-end
- Un desarrollador puede encargarse también del despliegue
- Una persona actúa como Product Manager y Product Owner
- Un diseñador cubre UX y UI
- Un DevOps Engineer administra nube, servidores y monitoreo

13.2 Empresa grande

- Existen varios equipos de desarrollo
- Hay especialistas dedicados a seguridad
- Platform Engineering y SRE pueden estar separados
- Puede haber arquitectos por dominio
- Los perfiles de QA y de datos se especializan por función

La especialización permite manejar sistemas complejos, pero aumenta la necesidad de coordinación entre equipos.

14. Clasificación general de los roles

Otra forma de mirar el ecosistema es agrupar los roles según su orientación principal:

Orientados al producto

- Product Manager
- Product Owner
- Business Analyst
- Customer Success

Orientados a los usuarios

- UX Researcher
- UX Designer
- UI Designer
- UX Writer
- Support Engineer

Orientados a la construcción

- Front-end Developer
- Back-end Developer
- Full-stack Developer
- Mobile Developer
- Game Developer
- Embedded Developer

Orientados a la calidad

- QA Manual
- QA Automation Engineer
- Performance Tester

Orientados a datos

- Data Engineer
- Data Analyst
- Data Scientist
- ML Engineer
- DBA

Orientados a infraestructura

- DevOps Engineer
- Cloud Engineer
- Platform Engineer
- SRE
- System Administrator

Orientados a seguridad

- Security Engineer
- AppSec Engineer
- DevSecOps Engineer

Orientados al liderazgo

- Software Architect
- Tech Lead
- Engineering Manager
- Scrum Master
- Release Manager

15. Conclusión

El desarrollo de software es una actividad multidisciplinaria. El programador ocupa un lugar central, pero no trabaja solo. Para crear un producto completo se necesita comprender el problema, diseñar la experiencia, escribir código, validar la calidad, administrar datos, desplegar la aplicación, mantenerla disponible, protegerla y acompañar a sus usuarios.

Producto define el valor.

Diseño define la experiencia.

Desarrollo crea la solución.

Arquitectura organiza la tecnología.

Calidad verifica el funcionamiento.

Datos transforma información en conocimiento.

DevOps y operaciones permiten ejecutar el sistema.

Seguridad protege el producto.

Soporte mantiene la relación con los usuarios.

Liderazgo coordina personas y decisiones.

El éxito de un proyecto no depende solamente de que cada especialista cumpla su tarea, sino también de la comunicación, la colaboración y la coordinación entre todas las áreas.